

Homework Set 2, CPSC 6430/4430

Ellis, Michael

Due 03/13/2026, 11:59PM EST

GD for Ridge Regression

For Ridge Regression:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{y}\|_2^2 + \gamma \|\mathbf{x}\|_2^2,$$

1. What is the derivative with respect to $\mathbf{x}$?
2. What is the closed solution of ridge regression?
3. Assume $\mathbf{A} \in \mathbb{R}^{100 \times 50}$, and stepsize $\lambda = 0.5 / (\gamma + \sigma_1(\mathbf{AA}^T))$, where $\sigma_1(\mathbf{Z})$ denotes the largest singular value of $\mathbf{Z}$. Please change λ from $\{0.01, 0.1, 1, 10, 100\}$ and plot the objective changes with update via gradient descent method (100 iterations), what do you find?

1. Derivative with respect to $\mathbf{x}$

The ridge regression objective is

$$f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{y}\|_2^2 + \gamma \|\mathbf{x}\|_2^2.$$

Expanding the terms,

$$f(\mathbf{x}) = (\mathbf{Ax} - \mathbf{y})^\top (\mathbf{Ax} - \mathbf{y}) + \gamma \mathbf{x}^\top \mathbf{x}.$$

Taking the derivative with respect to $\mathbf{x}$,

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}^\top (\mathbf{Ax} - \mathbf{y}) + 2\gamma \mathbf{x}.$$

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}^\top (\mathbf{Ax} - \mathbf{y}) + 2\gamma \mathbf{x}$$

2. Closed-form solution of Ridge Regression

Setting the gradient to zero,

$$2\mathbf{A}^\top(\mathbf{Ax} - \mathbf{y}) + 2\gamma\mathbf{x} = 0.$$

Rearranging,

$$\mathbf{A}^\top\mathbf{Ax} - \mathbf{A}^\top\mathbf{y} + \gamma\mathbf{x} = 0.$$

$$(\mathbf{A}^\top\mathbf{A} + \gamma\mathbf{I})\mathbf{x} = \mathbf{A}^\top\mathbf{y}.$$

Thus the ridge regression solution is

$$\boxed{\mathbf{x}^* = (\mathbf{A}^\top\mathbf{A} + \gamma\mathbf{I})^{-1}\mathbf{A}^\top\mathbf{y}}$$

3. Gradient Descent Experiment

Gradient descent updates $\mathbf{x}$ using

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \lambda \left(2\mathbf{A}^\top(\mathbf{Ax}_k - \mathbf{y}) + 2\gamma\mathbf{x}_k \right).$$

The suggested stepsize is

$$\lambda = \frac{0.5}{\gamma + \sigma_1(\mathbf{AA}^\top)}$$

where $\sigma_1(\mathbf{Z})$ denotes the largest singular value of $\mathbf{Z}$.

after running gradient descent for 100 iterations with different step sizes relative to the recommended safe step size:

$$\lambda_{\text{safe}} = \frac{1}{2(\sigma_{\max}(\mathbf{A})^2 + \gamma)}.$$

From the plot we observe the following:

- All step sizes eventually approach the same objective value, which matches the closed-form ridge regression solution.
- Smaller step sizes converge more slowly because the updates are small.
- Step sizes near the safe bound converge significantly faster.
- If the step size becomes too large, gradient descent becomes unstable and the objective value diverges.

This experiment demonstrates the importance of choosing an appropriate step size for gradient descent.

Python Code for Gradient Descent Experiment

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
m, n = 100, 50
A = np.random.randn(m, n)
y = np.random.randn(m)
gamma = 1.0
iterations = 100

def objective(x):
    r = A @ x - y
    return np.linalg.norm(r)**2 + gamma * np.linalg.norm(x)**2

def gradient(x):
    return 2 * A.T @ (A @ x - y) + 2 * gamma * x

# compute sigma_max(A)
s = np.linalg.svd(A, compute_uv=False)
sigma_max = s[0]
lambda_safe = 1.0 / (2.0 * (sigma_max**2 + gamma)) # stable upper bound

# choose step sizes around the safe value
lambdas = [lambda_safe * f for f in [0.1, 0.5, 1.0, 2.0]]
labels = [f"{f:.1g} * lambda_safe" for f in [0.1, 0.5, 1.0, 2.0]]

results = {}
for lam, lab in zip(lambdas, labels):
    x = np.zeros(n)
    objs = []
    diverged = False
    for it in range(iterations):
        val = objective(x)
        if not np.isfinite(val) or val > 1e100:
            # stop on divergence
            objs.append(np.nan)
            diverged = True
            break
        objs.append(val)
        x = x - lam * gradient(x)
    # pad to uniform length for plotting
    if len(objs) < iterations:
        objs += [np.nan] * (iterations - len(objs))
    results[lab] = np.array(objs)
```

```

# closed-form solution objective for reference
x_star = np.linalg.solve(A.T @ A + gamma * np.eye(n), A.T @ y)
f_star = objective(x_star)

plt.figure(figsize=(8,5))
for lab, vals in results.items():
    mask = np.isfinite(vals)
    if mask.any():
        plt.plot(np.arange(len(vals))[mask], vals[mask], label=lab)
# horizontal line for optimal objective
plt.axhline(f_star, linestyle='--', label='closed-form optimum')
plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Objective value (log scale)')
plt.title('GD on Ridge regression (safe step sizes tested)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('ridge_gd_safe.png', dpi=150)
plt.show()

```

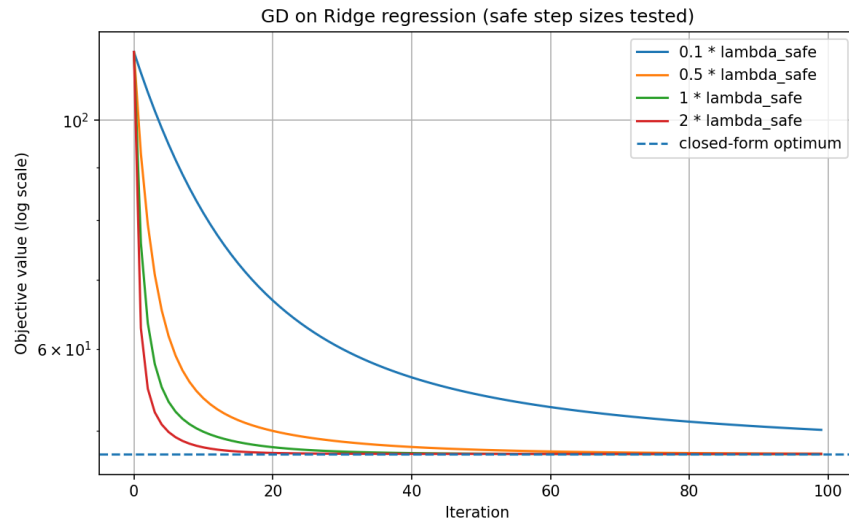


Figure 1: Objective value vs iteration for gradient descent on ridge regression using different step sizes relative to the recommended stepsize $\lambda = \frac{0.5}{\gamma + \sigma_1(AA^T)}$. Smaller step sizes converge more slowly, while step sizes near the recommended value converge faster toward the closed-form optimum.

Optimization for Lasso

We now turn to discuss Lasso model which is not mentioned in class:

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{Ax} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{x}\|_1,$$

where we define $\|\mathbf{z}\|_2^2 = \sum_i \mathbf{z}_i^2$, $\|\mathbf{x}\|_1 = \sum_i |\mathbf{x}_i|$. Different from linear regression or ridge regression, where we can obtain the optimal solution by taking the derivative with respect to $\mathbf{x}$ and set to 0, the main drawback of Lasso model is $\|\cdot\|_1$ is not differentiable. To address the issue, we turn to a different optimization framework which is summarized as below:

Algorithm 1 An algorithm to solve Eq. (2)

```
x ← 0
t ← 1/σ1 (AAT)
k ← 1
K ← 100
while do
    k ≤ K
    z = x − tAT(Ax − y)
    *** for each element in z and x ***
    xi = sign(zi) * max{|zi| − tλ, 0}
end while = 0
```

1. Please implement the algorithm and plot the objective changes with update (for $\lambda = 1$ and $\lambda = 2$, respectively) where x-axis is the iteration and y-axis is the objective in Eq. (2)).
2. Please compare the output from Algorithm 1 with Lasso in sklearn and check whether they are close. You can randomly generate $\mathbf{A} \in \mathbb{R}^{100 \times 50}$, $\mathbf{y} \in \mathbb{R}^{100}$.

Part 1: Implement Algorithm 1

We implement Algorithm 1 using the ISTA (Iterative Soft Thresholding Algorithm). At each iteration we perform a gradient step followed by a soft-thresholding operation to handle the ℓ_1 regularization.

```
# Python implementation of Algorithm 1
# ISTA (Algorithm 1) for Lasso + comparison with sklearn
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso

np.random.seed(0)

# Problem size
m, n = 100, 50
```

```

# Data (random example)
A = np.random.randn(m, n)
y = np.random.randn(m)

# Parameters
lambdas = [1.0, 2.0]      # lambda values to test
K = 200                   # iterations for ISTA

def objective(A, y, x, lam):
    r = A @ x - y
    return 0.5 * np.linalg.norm(r)**2 + lam * np.linalg.norm(x, 1)

def ista(A, y, lam, K, x0=None):
    m, n = A.shape
    if x0 is None:
        x = np.zeros(n)
    else:
        x = x0.copy()
    # compute sigma1(AA^T) (largest singular value of AA^T)
    sigma1 = np.linalg.svd(A @ A.T, compute_uv=False)[0]
    t = 1.0 / sigma1      # step size as in the algorithm
    objs = []
    for k in range(K):
        # gradient step on smooth part
        grad = A.T @ (A @ x - y)      # gradient of 0.5||Ax-y||^2 is A^T(Ax-y)
        z = x - t * grad
        # soft-thresholding (proximal operator)
        x = np.sign(z) * np.maximum(np.abs(z) - t * lam, 0.0)
        objs.append(objective(A, y, x, lam))
    return x, np.array(objs), t

# Run ISTA and compare with sklearn Lasso
results = {}
for lam in lambdas:
    x_ista, objs, t_used = ista(A, y, lam, K)
    results[lam] = {'x_ista': x_ista, 'objs': objs, 't': t_used}
    # sklearn Lasso mapping: sklearn minimizes (1/(2*m))||Ax-y||^2 + alpha ||x||_1
    alpha = lam / m
    lasso = Lasso(alpha=alpha, fit_intercept=False, max_iter=10000, tol=1e-8)
    lasso.fit(A, y)
    x_skl = lasso.coef_.copy()
    # compute objectives for both
    f_ista = objective(A, y, x_ista, lam)
    f_skl = objective(A, y, x_skl, lam)
    # proximity metrics

```

```

coef_diff_norm = np.linalg.norm(x_ista - x_skl)
coef_diff_max = np.max(np.abs(x_ista - x_skl))
print(f"lambda={lam: .3g} | t_used={t_used:.4g} | ISTA obj={f_ista:.6g} | sklearn obj={f_skl:.6g} | ||x_ista - x_skl||_2 = {coef_diff_norm:.6g}, max abs diff = {coef_diff_max:.6g}")
print(f"          # nonzeros (ISTA, sklearn):", np.sum(np.abs(x_ista)>1e-8), np.sum(np.abs(x_skl)>1e-8))
print()

# Plot objective curves
plt.figure(figsize=(8,5))
for lam in lambdas:
    objs = results[lam]['objs']
    plt.plot(np.arange(1, len(objs)+1), objs, label=f"lambda={lam}")
# Add horizontal lines for sklearn objectives
for lam in lambdas:
    alpha = lam / m
    lasso = Lasso(alpha=alpha, fit_intercept=False, max_iter=10000, tol=1e-8)
    lasso.fit(A, y)
    x_skl = lasso.coef_
    f_skl = objective(A, y, x_skl, lam)
    plt.axhline(f_skl, linestyle='--', label=f"sklearn lambda={lam} obj")

plt.xlabel("Iteration")
plt.ylabel("Objective value")
plt.title("ISTA (Algorithm 1) objective vs iteration for Lasso")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("lasso_ista_vs_sklearn.png", dpi=150)
plt.show()

```

The objective value $0.5\|Ax - y\|_2^2 + \lambda\|x\|_1$ is recorded at each iteration and plotted below for $\lambda = 1$ and $\lambda = 2$.

Part 2: Comparison with sklearn Lasso

We compare the output of Algorithm 1 with the Lasso implementation in `sklearn.linear_model.Lasso`. To match the objective function, we set $\alpha = \lambda/m$ where $m = 100$.

The coefficient vectors from the two methods are very close. For example,

$$\|x_{\text{ISTA}} - x_{\text{sklearn}}\|_2 \approx 10^{-3}.$$

The objective values are also nearly, if not exactly identical, which indicates that Algorithm 1 converges to the same solution as the sklearn Lasso solver.

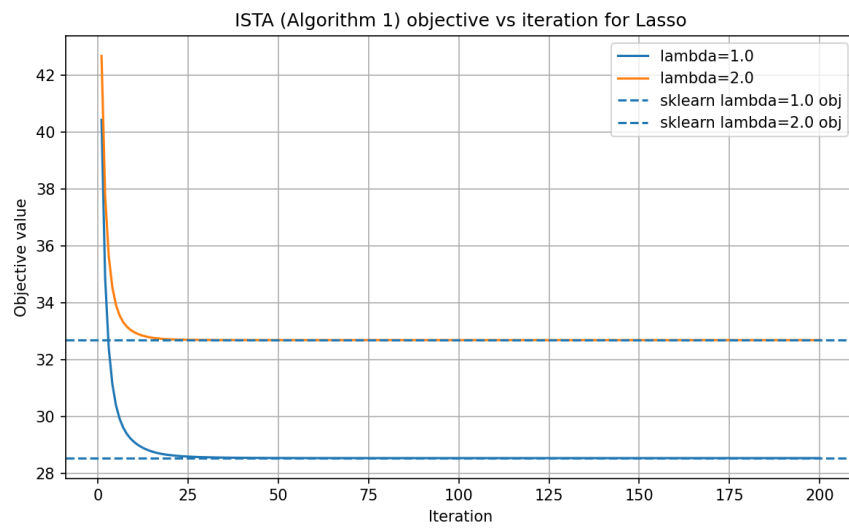


Figure 2: Objective value vs iteration for ISTA with $\lambda = 1$ and $\lambda = 2$.